

Week 11 Project Guide

Computer Vision with Deep Learning

Role Requirements · Notebook Walkthrough · Submission Guide

Deep Learning · Fusemachines AI Fellowship · Susan Ghimire

How to use this document: Read **Your Role** first — every modelling decision in the notebook flows from the business framing. Use the **Notebook Walkthrough** as a per-part checklist. Check the **Submission Requirements** carefully before uploading — a missing artefact or an empty Reflect cell will fail the submission. For background reading, videos, and papers, see the companion Resource Guide. All links are clickable.

1 YOUR ROLE

The business scenario you are operating in for this entire assignment

You are a **Computer Vision Engineer at QuickVision AI**, a logistics-tech company operating cameras at **500 warehouse locations**. Your team's current system is a rule-based HSV colour-segmentation pipeline built in Lab 1. It achieves only **~45% accuracy** on real-world footage — colour alone cannot distinguish between product SKUs of similar hue, and it fails completely on new product types not seen during hand-tuning. Your Director of

Engineering asks two questions: (1) “Which model family should we deploy for reliable product classification across 500 cameras?” and (2) “How do we onboard a new product type in hours, not weeks, when we have zero labelled images of it?”

Your task: work through five phases of the modern computer vision stack — CNN classification, object detection, image segmentation, generative augmentation, and vision transformers — on CIFAR-10 and COCO-pretrained models, then produce a deployment memo that answers both questions above with cited accuracy and latency numbers.

Phase	Your deliverable	Business goal
1 — CNN	ResNet-50 classifier + GradCAM	Beat the Lab 1 HSV 45% baseline
2 — Detection	IoU + NMS + Faster R-CNN demo	Locate individual items in a scene
3 — Segmentation	DeepLabv3+ + mIoU metrics	Pixel-level product/zone mapping
4 — Generative	VAE training + latent interpolation	Synthetic data for low-data SKUs
5 — ViT + Deploy	PatchEmbedding + CLIP + ONNX export	Zero-shot onboarding for new SKUs

Key challenge: A single model family cannot satisfy both of the Director's questions. High-accuracy supervised classification (ResNet-50) requires hundreds of labelled images per class — it cannot answer the zero-label onboarding question. Zero-shot models (CLIP) answer onboarding instantly but at lower accuracy than a fine-tuned classifier. Your deployment memo (Part 5) must reconcile this trade-off, not just report the highest single accuracy number.

2 DATASET

CIFAR-10 (primary) + COCO-pretrained models — no manual download required

Property	Value	Notes
Source	torchvision.datasets .CIFAR10	Auto-downloads to ../data on first run (~170 MB).
Training / test images	50 000 / 10 000	You use 5 000 train / 1 000 test in Part 1 for speed.
Image size	32 × 32 pixels, RGB	Resized to 224 × 224 for ResNet-50 and DeepLabv3+ (ImageNet-pretrained).
Classes	10	airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
Detection / segmentation weights	COCO-pretrained	Faster R-CNN and DeepLabv3+ load pretrained weights automatically (~160 MB) — used in eval mode, no retraining required.

Dataset loading:

```
import torchvision, torchvision.transforms as T
from torchvision import datasets, models

raw_train = datasets.CIFAR10('../data', train=True, download=True)
raw_test = datasets.CIFAR10('../data', train=False, download=True)

# COCO-pretrained detection / segmentation models — no download step needed:
frcnn = models.detection.fasterrcnn_resnet50_fpn(weights='DEFAULT').eval()
seg_model = models.segmentation.deeplabv3_resnet50(weights='DEFAULT').eval()
```

Why CIFAR-10 + COCO-pretrained? CIFAR-10 is small enough to train a full ResNet-50 head and a VAE from scratch within a session, while still being a genuine 10-class problem — not a toy. Faster R-CNN and DeepLabv3+ are used purely for inference: COCO's 80 classes and 200K images make from-scratch retraining unnecessary for this module's learning goals, and mirrors how most production CV teams start — adapt a pretrained model before ever training one from scratch.

3 NOTEBOOK WALKTHROUGH

Part-by-part guide for completing the assignment

Part 1 — CNN Foundations & Classification (Q1–Q5)

Q1 (Output-size formula)

Apply $\text{out} = (W - K + 2P) / S + 1$ to five configurations including the ResNet-50 stem (7×7, stride 2). Two SELF-CHECKs verify your formula.

Q2 (ResNet-50 transfer learning setup)

Resize CIFAR-10 to 224 and normalise with ImageNet statistics. Freeze all backbone parameters, then replace `model.fc` with `nn.Linear(2048, 10)` — only the new head (~20K params) should be trainable.

Q3 (Training loop)

Implement the standard train-and-eval loop for 3 epochs: forward pass, loss, `backward()`, `step()`, prediction extraction. SELF-CHECK requires val accuracy > 60%.

Q4 (GradCAM)

Register a forward hook and a `full_backward` hook on `model.layer4[-1]`. Compute per-channel weights from gradients, then a ReLU-clipped weighted sum of activations to produce the class activation map.

Q5 (Reflect)

Report your val accuracy, trainable parameter count, a GradCAM observation, and the conceptual link between learned convolution kernels and Lab 2's hand-built Sobel kernels.

Part 2 — Object Detection (Q6–Q9)

Q6 (IoU from scratch)

Implement `compute_iou(box_a, box_b)` using the `[x1,y1,x2,y2]` format. Clamp intersection width/height to zero when boxes do not overlap.

Q7 (Faster R-CNN inference)

Load `fasterrcnn_resnet50_fpn(weights='DEFAULT')` in eval mode. Pass images as a Python list (not a batch tensor) and visualise the returned boxes, labels, and scores.

Q8 (NMS from scratch)

Sort boxes by score descending, greedily keep the top box, and suppress any remaining box with $\text{IoU} \geq \text{threshold}$ against it.

Q9 (Reflect)

Report detection counts, your NMS result, the conceptual link to Lab 3.2's Hough accumulator peak-picking, and a use case favouring Faster R-CNN over YOLO.

Part 3 — Image Segmentation (Q10–Q13)

Q10 (DeepLabv3+ semantic segmentation)

Load `deeplabv3_resnet50(weights='DEFAULT')`. Take `argmax(1)` over the 21 VOC classes from the model's 'out' key to produce the pixel mask.

Q11 (pixel_accuracy and mean_iou)

Implement both metrics from scratch with NumPy boolean masks. Skip classes absent from the ground truth to avoid division by zero.

Q12 (U-Net diagram)

Diagram the encoder–bottleneck–decoder structure, label spatial sizes and channel counts, and explain the role of skip connections.

Q13 (Reflect)

Report classes detected, your pixel_accuracy result, a scenario where a classical (Lab 3.1-style) pipeline beats deep segmentation, and the distinction between semantic and instance segmentation.

Part 4 — Generative Models — VAE (Q14–Q16)

Q14 (VAE architecture + training)

Complete the `reparameterise` method: `std = exp(0.5 * logvar)`, `eps = randn_like(std)`, return `mu + std * eps`. Complete the ELBO loss: summed-MSE reconstruction term plus the closed-form KL divergence.

Q15 (Reconstruction + latent interpolation)

Use `torch.linspace(0, 1, n_steps)` for interpolation weights α , then decode $z_{\text{interp}} = (1-\alpha) \cdot z_0 + \alpha \cdot z_1$ for each α and display the resulting image grid.

Q16 (Reflect)

Report your loss values, explain why VAE reconstructions are blurry, describe interpolation smoothness, and outline a VAE-based augmentation workflow for a 50-image low-data SKU.

Part 5 — Vision Transformers & Deployment (Q17–Q20)

Q17 (PatchEmbedding)

Set `num_patches = (img_size // patch_size) ** 2`. Use a `Conv2d` with `kernel_size=stride=patch_size` as the patch projection, then flatten and prepend the [CLS] token.

Q18 (CLIP zero-shot classification)

Build text prompts ("a photo of a {class}"), L2-normalise both image and text features, then classify by cosine similarity — `img_feat @ text_feats.T`.

Q19 (ONNX export)

Move the model to CPU before exporting. Call `torch.onnx.export` with `dynamic_axes` on the batch dimension and `external_data=False` (recent PyTorch versions otherwise split large models into a separate `.onnx.data` file), then verify with `onnxruntime.InferenceSession`.

Q20 (Comparison table + deployment memo)

Produce a model comparison table (Lab 1 HSV, ResNet-50, CLIP) and write the deployment memo that answers the Director's two questions from Section 1, citing your actual accuracy and latency numbers.

4 SUBMISSION REQUIREMENTS

Exactly what to submit, how, and what will be checked

4.1 Files to Submit

File	Description	Required?
<code>W11_CV_Assignment_Notebook.ipynb</code>	Your completed assignment notebook. Fully executed — all 20 questions answered, all plots visible, all SELF-CHECK cells pass, all Reflect cells filled in. Run top-to-bottom with no errors before submitting.	✓ Required
<code>resnet50_cifar10.onnx</code>	The exported ONNX model from Q19 — a single self-contained file (export with <code>external_data=False</code>). Must reload with <code>onnxruntime</code> and produce logits of shape (1, 10) on a dummy input. This is the deployable artefact for the Director's memo.	✓ Required
Deployment Memo (inline in Q20)	Memo answering both of the Director's questions from Section 1: which model to deploy for production classification, and how to onboard a new zero-label SKU — citing actual accuracy and latency figures.	✓ Required

4.2 Notebook Quality Standards

- All 20 questions answered — Q19 (ONNX export) and Q20 (deployment memo) are required.
- All cells executed top-to-bottom — no skipped cells, no `NameError` or `ImportError`.
- Every `= None` line marked with a `# TODO` comment replaced with your computed value — each is a named fill-in; leaving any as `None` will cause a runtime error. (A few unmarked `None` values — e.g. GradCAM's initial activation/gradient cache — are intentional scaffold state, not fill-ins.)
- Every Reflect cell contains your own reasoning with specific numbers — not a copy from the session notebook and not an unedited AI-generated answer.
- All SELF-CHECK assert cells pass without error.
- Q3 reports `val accuracy ≥ 60%` at epoch 3, with the actual number stated in Reflect.
- Q11 `pixel_accuracy` equals 1.0 and `mean_iou` is ~ 1.0 (within $1e-4$) on the perfect-mask SELF-CHECK test case — `mean_iou`'s epsilon-stabilised denominator means it never hits exactly 1.0.
- Q17 PatchEmbedding output shape is exactly (4, 65, 256) for a 32×32 image with 4×4 patches.
- Q19 onnx file loads with `onnxruntime` and produces logits of shape (1, 10).
- Q20 deployment memo cites your actual accuracy and latency numbers — no placeholder [brackets] remaining.

Do not submit: The CIFAR-10 data directory, the session solution notebook, downloaded pretrained-weight cache files, `__pycache__`, or `.ipynb_checkpoints` folders. The only binary artefact beyond the notebook is `resnet50_cifar10.onnx`.

5 COMMON MISTAKES TO AVOID

Patterns that produce wrong results or fail the submission check

X Wrong resize in Q2

`T.Resize((32, 32))` (a tuple) keeps images at CIFAR-10 size. ResNet-50 expects 224×224 and will produce incorrect feature shapes downstream. Use `T.Resize(224)` (an integer) — torchvision resizes the shorter edge to 224, producing a square 224×224 image from a square input.

X Forgetting to freeze parameters before replacing the FC head

If you replace `model.fc` before freezing, then loop for `p in model.parameters()`: `p.requires_grad = False`, you also freeze the new FC head — it will never learn. Freeze first (all layers), then assign `model.fc = nn.Linear(2048, 10)`. The new module has `requires_grad=True` by default.

X Passing a batch tensor to Faster R-CNN

`frCNN(imgs)` where `imgs` has shape $(1, 3, H, W)$ raises a `TypeError` — Faster R-CNN does not accept a batch tensor. Pass a Python list instead: `frCNN([img_tensor])`. The output is also a list of dicts.

X Using `reduction='mean'` in the VAE reconstruction loss

`F.mse_loss(x_hat, x, reduction='mean')` divides by $H \times W \times C \times N$, making the reconstruction loss thousands of times smaller than expected. This lets the KL term dominate and the model collapses to producing near-uniform noise. Use `reduction='sum'` then divide by `x.shape[0]` (batch size only).

X Exporting ONNX with the model still on GPU

`torch.onnx.export` with a CUDA model and a CPU dummy input raises a `RuntimeError`: input and model must be on the same device. Move the model to CPU before export with `model.cpu()`; the exported ONNX file is device-agnostic regardless of where you trained it.

X Submitting resnet50_cifar10.onnx without external_data=False

Recent PyTorch versions (2.5+) default to splitting large model weights into a separate `resnet50_cifar10.onnx.data` file next to the small graph-only `.onnx` file. The two files load together fine on your machine, but if you submit or copy only the `.onnx` file, `onnxruntime.InferenceSession` will fail with an external data path error — the model is silently incomplete without its companion file. Pass `external_data=False` to `torch.onnx.export` to keep everything in one self-contained file, matching what the submission checklist expects.

X Not L2-normalising CLIP features before cosine similarity

`img_feat @ text_feats.T` computes raw dot products, not cosine similarity, unless both feature vectors are unit-normalised first. Unnormalised results are dominated by feature magnitude rather than semantic direction. Always normalise: `feat = feat / feat.norm(dim=-1, keepdim=True)` for both image and text features before the dot product.

Remember: The Director's question is not 'which model has the highest accuracy?' — it is 'what should we actually deploy, given that one model must classify at scale and another must onboard new products with zero labels?' A memo that reconciles both constraints with real numbers is more valuable than a single leaderboard entry. A thoughtful deployment recommendation that names a specific trade-off you accepted is worth more than the highest CIFAR-10 accuracy in the room.